In [ ]:
```
# Day - 1 - 15/06/2026
#Method 1
#Program to check whether the given number is prime or not
def isPrime(n):
    if n == 1:
        return False
    for f in range(2,n):
        if n % f  == 0:
            return False
    return True

# main function
x = int(input())
if isPrime(x):
    print("Prime")
else:
    print("Not Prime")
```

In [ ]:
```
# print(*range(10,2,-2)) # * unpacking the sequence
# for i in range(10):
#     print(i)
```

In [ ]:
```
def isPrime(n):
    if n == 1:
        return False
    for f in range(2,n//2+1):
        if n % f  == 0:
            return False
    return True

# main function
x = int(input())
if isPrime(x):
    print("Prime")
else:
    print("Not Prime")
```

In [ ]:
```
import math as m
def isPrime(n):
    if n == 1:
        return False
    for f in range(2,int(m.sqrt(n))+1):
        if n % f  == 0:
            return False
    return True

# main function
x = int(input())
if isPrime(x):
    print("Prime")
else:
    print("Not Prime")
#     7 ==> 6 x 1 + 1
#     11 ==> 6 x 2 - 1
#     13 ==> 6 x 2 + 1
#     23 ==> 6 x 4 - 1
# All prime numbers are either 6N + 1 or 6N -1 except 2 & 3
# But all 6N+1 and 6N-1 are not prime numbers
```

In [ ]:

In [ ]:

In [ ]:
```
#Method - 4
import math as m
def isPrime(n):
    if n == 1:
        return False
    elif n > 3 and (n % 2 == 0 or n % 3 == 0):
        return False
    else:
        for f in range(5,int(m.sqrt(n))+1, 6):
            if n % f  == 0 or n % (f + 2) == 0:
                return False
        return True

# main function
x = int(input())
if isPrime(x):
    print("Prime")
else:
    print("Not Prime")
```

In [ ]:
```
# Day - 2  - 16/06/2026
# Program to count number of vowels in a string
def isVowel(char):
    return char.upper() in "AEIOU"

cnt = 0
s1 = input()
for c in s1:
    if isVowel(c):
        cnt += 1
print(cnt)
```

In [ ]:
```
# program to reverse a string
s = "deepika"
s2 = s[::-1] # slicing
print (s2)
```

In [ ]:
```
s1 = input()
s2 = ""
n = len (s1)
for index in range(n-1, -1, -1):
    s2 += s1[index]
print(s2)
```

In [ ]:
```
# Two pointer approach to reverse string
s = input()
s1 = list(s) # converting string into list
n = len(s1)
left,right = 0, n-1
while left < right:
    s1[left], s1[right] = s1[right], s1[left]
    left += 1
    right -=1
print("".join(s1)) # converting list into string
```

In [ ]:
```
l1 = ["a", "a", "s", "a", "i", "1"]
print("".join(l1))
```

In [ ]:
```
# Program to sort elements using selection sort - greedy method
n = int(input())
# string elements
# names = input().split() [:n]
names = input().split()
# for i in range(n):
#     names[i] = int(names[i])
#implementing selection sort algorithm - descending order
for i in range(n-1):
    #assuming ith element is max
    maxIndex = i
    for j in range(i+1, n):
        #chekcing whether max element present in the remaining list
        if names[maxIndex] < names[j]:
                # replace max value index if max value present in the remaining list
                maxIndex = j
    # swap ith element and maxIndex element if ith element is not max
    if i !=maxIndex:
        names[i], names[maxIndex] = names[maxIndex], names[i]

print (names)
```

In [ ]:
```
l1 = list(map(int, input().split()))
print(l1)
```

In [ ]:
```
# Activity selection problem
n = int(input())
activities = []
for _ in range(n):
    activities.append(list(map(int, input().split(","))))
# sorting the activities based on end time of activity - Ascending order
activities.sort(key=lambda x: x[1])
result = []
result.append(activities[0])
i = 0
while (i < n):
    for j in range(i+1,n):
        if activities[j][0] >= activities[i][1]:
            result.append(activities[j])
            i = j;
    i+=1
print(result)
```

In [ ]:
```
l1 = [10,20,30]
# unpacking list
x,y = l1
print(x, y)
```

In [ ]:
```
l1 = [7,4,9,11,2,17,14]
l1.sort(reverse=True)
print(l1)
```

In [ ]:
```
l1 = [7,4,9,11,2,17,14]
# sorted_list = sorted(l1) # Ascending order
sorted_list = sorted(l1, reverse=True) # descending order
print(l1)
print(sorted_list)
```

In [ ]:
```
l1 = [[0,6], [1,5],[8,11], [2,6],[1,4]]
l1.sort(key=lambda x: x[1])
print(l1)
```

In [ ]:
```
# Program to implement coin change problem
amt = int(input())
n = int(input())
coins = list(map(int, input().split())) # 100 500 200 10 50
totalCoins = 0
coins.sort(reverse=True)
if amt >= coins[n-1]:
    for coin in coins:
        if amt>= coin:
            totalCoins += amt // coin
            amt %= coin
        if amt == 0:
            break
    if amt == 0:
        print(totalCoins)
    else:
        print(-1)
else:
    print(-1)
```

In [ ]:
```
# Day - 3  - 17/06/2026
# Program to implement fractional knapsack problem
m = int(input())
n = int(input())
products = [] # products = list()
for _ in range(n):
    profit, weight = list(map(int,input().split()))
    ratio = profit / weight
    products.append([ratio, profit, weight])
products.sort(reverse=True)
maxProfit = 0
index = 0
while m > 0 and index < n:
    if m >= products[index][2]:
        maxProfit += products[index][1]
        m -= products[index][2]
    else:
        maxProfit += products[index][0] * m
        m = 0
    index+=1
print(maxProfit)
```

In [ ]:
```
#Program to find factorial of N
import math as m
n = int(input())
print(m.factorial(n))
```

In [ ]:
```
#Program to find factorial of N using iteration
n = int(input())
fact = 1
# iterative method
for f in range(2,n+1):
    fact = fact * f
print(fact)
```

In [ ]:
```
#Program to find factorial of N using recursion
def fact(n):
    if n == 0: # base case
        return 1
    return n * fact(n-1) # recursive call



n = int(input()) # 5
print(fact(n))

# def fact(5):
#     if 5 == 0: # base case
#         return 1
#     return 5 * fact(5-1) # recursive cal

# def fact(4):
#     if 4 == 0: # base case
#         return 1
#     return 4 * fact(4-1) # recursive cal

# def fact(3):
#     if 3 == 0: # base case
#         return 1
#     return 3 * fact(3-1) # recursive cal

# def fact(2):
#     if 2 == 0: # base case
#         return 1
#     return 2 * fact(2-1) # recursive cal

# def fact(1):
#     if 1 == 0: # base case
#         return 1
#     return 1 * fact(1-1) # recursive cal

# def fact(0):
#     if 0 == 0: # base case
#         return 1
```

In [ ]:
```
# Day - 4
# Program to print natural numbers upto N using iterative method
# 5 --> 1 2 3 4 5
n = int(input())
for i in range(1, n+1):
    print (i, end=" ")
```

In [ ]:
```
print(*range(1,int(input())+1), sep=",")
```

In [ ]:
```
# Program to print natural numbers upto N using recursion method
# 5 --> 1 2 3 4 5
def printf(n):
    if n == 0:
        return
#     printf(n-1) # head recursion
    print(n, end=" ")
    printf(n-1)  # tail recursion


n = int(input())
printf(n)
```

In [ ]:
```
##### Program to demonstrate double recursion
def printf(n):
    if n == 0:
        return
    printf(n-1) # head recursion
    print(n, end=" ")
    printf(n-1) # tail recursion

# main function
printf(3)
```

In [2]:

```
# Program to print nth term of fibonacci series
cnt = 0
def fib(n):
    global cnt
    cnt+=1
    # base case
    if n <= 1:
        return n
    return fib(n-1)+fib(n-2)

# main function
n = int(input())
print(fib(n))
print (cnt)
```

```
5
5
15
```

In [13]:

```
# Program to print nth term of fibonacci series using memoization - dp method
cnt = 0
def fib(n, dp):
    global cnt
    cnt+=1
    # base case
    if n == 0 or n == 1:
        return n
    if dp[n]!=-1:
        return dp[n]
    dp[n] = fib(n-1, dp) + fib(n-2, dp)
    return dp[n]

# main function
n = int(input())
dp = [-1] * (n+1)  # repeat [-1] n times
print(fib(n, dp))
print(cnt)
```

```
5
5
9
```

In [7]:

```
# print ("AjithKumar " * 100)
print ([-1] * 100)
```

```
[-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -
1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1,
-1, -1, -1, -1, -1, -1, -1, -1, -1, -1, -1]
```

In [14]:

```
# Program to find nth term of fibonacci series - using tabulation
def fib(n):
    dp = [0] * (n+1)
    dp[1] = 1
    for i in range(2, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]

# main function
n = int(input())
print(fib(n))
```

```
6
8
```

In [18]:

```
# Program to count the number of way to climb the staircase
def countWays(n):
    dp = [0] * (n+1)  # creating dp table with n+1 size
    dp[1],dp[2] = 1,2
    for i in range(3, n+1):
        dp[i] = dp[i-1] + dp[i-2]
    return dp[n]


#main function
steps = int(input())
print(countWays(steps))
```

```
7
21
```

In [25]:
```
#coin change using DP
def minimumCoins(amt, coins, n):
    dp = [amt+1] * (amt+1)
    dp[0] = 0
    for coin in coins:
        for a in range(coin, amt+1):
            dp[a] = min(dp[a], 1+dp[a-coin])
    # checking whether found the change or not
    if dp[amt]!= amt+1:
        return dp[amt] # if yes return the minimum coin
    return -1 # if no return -1

amt = int(input())
n = int(input())
coins = list(map(int, input().split()))
print(minimumCoins(amt, coins,n))
```

```
11
3
6 4 2
-1
```

In [26]:
```
# Program to find length longest increasing subsequence using DP Method
def lis(data, n):
    # creating list with n elements initial as 1
    dp = [1] * n
    for i in range(1, n):
        for j in range(i):
            if arr[i]>arr[j]:
                dp[i] = max(dp[i], 1+dp[j])
    return max(dp)

# main function
n = int(input())
arr = list(map(int, input().split()))
print(lis(arr, n))
```

```
9
10 22 9 33 21 50 41 60 80
6
```

In [22]:
```
# Day - 5
# Program to implement 0/1 knapsack problem - DP
#knapsack Size
m = int(input())
#Product size
n = int(input())
products = []
dp = [[0] * (m+1) for _ in range(n+1)]
for _ in range(n):
    products.append(input().split())
for i in range(1,n+1):
    products[i-1][1], products[i-1][2] = int(products[i-1][1]), int(products[i-1][2])
    for j in range(1, m+1):
        if j >= products[i-1][2]:
            dp[i][j] = max(dp[i-1][j], products[i-1][1] + dp[i-1][j-products[i-1][2]])
        else:
            dp[i][j] = dp[i-1][j]
print(dp)
print(dp[n][m])
```

```
4
5
Mobile 1500 1
Suit 3000 4
Watch 2000 3
Laptop 3000 2
Ring 2000 1
[[0, 0, 0, 0, 0], [0, 1500, 1500, 1500, 1500], [0, 1500, 1500, 1500, 3000], [0, 1500, 1500, 2000, 3500], [0, 1500, 300
0, 4500, 4500], [0, 2000, 3500, 5000, 6500]]
6500
```

In [6]:
```
# Program to demonstrate two dimensional array
# m = 5
# n = 4
# dp = [[0] * n] * m
# print(dp)
# for row in dp:
#     print (id(row))
```

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
1329235705024
1329235705024
1329235705024
1329235705024
1329235705024
```

In [8]:
```
# n = int(input())
# for _ in range(n):
#     print("Harini", end=" ")
```

```
10
Harini Harini Harini Harini Harini Harini Harini Harini Harini Harini
```

In [10]:
```
l1 = list(map(int, input().split()))
# print(l1)
sqr_list = []
for element in l1:
    sqr_list.append(element * element)
print (sqr_list)
```

```
1 2 3 4 5 6 7
[1, 4, 9, 16, 25, 36, 49]
```

In [12]:
```
# Program to demonstrate list comprehension
l1 = list(map(int, input().split()))
#list comprehension
print( [element * element for element in l1])
# print(sqr_list)
```

```
1 2 3 4 5
[1, 4, 9, 16, 25]
```

In [13]:
```
print( [element * element for element in list(map(int, input().split()))])
```

```
1 2 3 4 5
[1, 4, 9, 16, 25]
```

In [16]:
```
m = 5
n = 4
# creating two dimensional array dynamically using list comprehension
dp = [[0] * n for _ in range(m)]
print(dp)
# dp[0][0] = 1
# print(dp)
for row in dp:
    print (id(row))
```

```
[[0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0], [0, 0, 0, 0]]
1329235615616
1329235456512
1329234898944
1329235698816
1329235391488
```

In [25]:
```
# Program to find length of longest common substring - DP
s1, s2 = input().split()
len1, len2 = len(s1), len(s2)
dp = [[0] * (len1+1) for _ in range(len2+1)]
for i in range(1,len2+1):
    for j in range(1, len1+1):
        if s2[i-1] == s1[j-1]:
            dp[i][j] = dp[i-1][j-1] + 1

print(max([max(row) for row in dp]))
```

```
hish vista
2
```

In [28]:
```
# Program to find length of longest common subsubsequence - DP
s1, s2 = input().split()
len1, len2 = len(s1), len(s2)
dp = [[0] * (len1+1) for _ in range(len2+1)]
for i in range(1,len2+1):
    for j in range(1, len1+1):
        if s2[i-1] == s1[j-1]:
            dp[i][j] = dp[i-1][j-1] + 1
        else:
            dp[i][j] = max(dp[i-1][j], dp[i][j-1])

print(max([max(row) for row in dp]))
```

```
fosh fort
2
```